

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
WORK INTEGRATED LEARNING PROGRAMMES (WILP) DIVISION

SECOND SEMESTER OF ACADEMIC YEAR 2025-2026

DSECLZG628T : DISSERTATION OUTLINE

STUDENT ID No.	2024DA04378
NAME OF THE STUDENT	RAJPUT DHIRAJ SING RAJESH SING
STUDENT'S EMAIL ADDRESS	2024da04378@wilp.bits-pilani.ac.in
STUDENT'S EMPLOYING ORGANIZATION & LOCATION	Cognizant Technology Solutions, Hyderabad
SUPERVISOR'S NAME	ROSHINI V
SUPERVISOR'S EMPLOYING ORGANIZATION & LOCATION	Cognizant Technology Solutions, Chennai
SUPERVISOR'S EMAIL ADDRESS	roshini.vijay2601@gmail.com
ADDITIONAL EXAMINER'S NAME	NIVEDITHA D S
ADDITIONAL EXAMINER'S EMPLOYING ORGANIZATION & LOCATION	Cognizant Technology Solutions, Bangalore
ADDITIONAL EXAMINER'S EMAIL ADDRESS	dsniveditha98@gmail.com
DISSERTATION TITLE	SECURE AI PIPELINE FOR SENSITIVE DATA

Dissertation Title:

SECURE AI PIPELINE FOR SENSITIVE DATA

Course No. : DSECLZG628T

Course Title: Dissertation

Dissertation Done by:

Student Name: RAJPUT DHIRAJ SING RAJESH SING

BITS ID: 2024DA04378

Degree Program: M.Tech. in Data Science and Engineering

Research Area: Artificial Intelligence — Privacy-Preserving Machine Learning

Dissertation Work carried out at:

Cognizant Technology Solutions, Hyderabad



**BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
VIDYA VIHAR, PILANI, RAJASTHAN - 333031.**

April 2026

Contents

1. Broad Area of Work	4
1.1 Elevator	4
2. Background and Problem Statement	4
2.1 Problem Identified and Solution Proposed	5
3. Objectives	5
4. Scope of Work	6
4.1 Form of the deliverable.	7
5. Tools, Technologies and System Architecture	7
5.1 Tools and their role	7
5.2 System architecture	8
6. Plan of Work and Measurable Milestones	9
6.1 Phase plan	9
6.2 Measurable milestones — Mid-Semester VIVA (target by 21 June 2026)	10
6.3 Measurable milestones — Final VIVA (target by 02 August 2026)	10
7. Literature References	11
8. Particulars of the Supervisor and Examiner	12
9. Remarks of the Supervisor	12

1. Broad Area of Work

This dissertation sits in the area of Artificial Intelligence, specifically privacy-preserving machine learning and secure ML system design. Most enterprise ML systems in production today were built before regulations like the DPDP Act and GDPR were in force, and most existing pipelines treat privacy and security as things to add on later, after the model is already trained and deployed. The work proposed here takes the opposite view: that privacy and security have to be designed into the pipeline from the start, not bolted on. The project covers the following areas:

- Differential Privacy: using DP-SGD during training so the resulting model does not leak information about individual records in the training set.
- De-identification: classifying attributes, applying k-anonymity, and encrypting raw data at rest.
- Secure deployment: an authenticated inference API with role-based access and an audit log that resists tampering.
- Attack evaluation: running a black-box membership inference attack against each pipeline configuration to measure how much privacy each defence actually provides.
- Layered defence study: a controlled comparison of k-anonymity, differential privacy, and the two together, to find out whether combining them gives stronger privacy than either one alone.

1.1 Elevator Pitch

Banks and hospitals run AI models on people's personal data, but those models can leak information about the very people they were trained on. This project builds a working pipeline that protects that data at every stage — from how it is stored, to how it is used to train the model, to how the model is queried — and then runs an actual attack against four versions of the pipeline to measure how much each protection helps. The deliverable is a clear, evidence-backed answer for engineers building real systems: which combination of privacy controls is worth the cost in model accuracy, and which is not.

2. Background and Problem Statement

Banks, hospitals, and insurers are putting more of their decision-making into machine learning models. The same period has seen a wave of new regulation: India's DPDP Act of 2023, GDPR in Europe, and HIPAA in the United States all set rules for how personal data can be collected, stored, and used. The problem is that a model trained the standard way still keeps traces of the people in its training data. Even when the raw dataset is never released, an attacker with query access to the model can often work out whether a specific person was part of the training set, or recover sensitive attributes about them.

A number of techniques exist to deal with this. k-anonymity, introduced by Sweeney in 2002, generalises and suppresses fields in a dataset so each record is indistinguishable from at least k-1 others. Differential Privacy, formalised by Dwork and adapted to deep learning by Abadi and colleagues in 2016 through DP-SGD, gives a mathematical bound on how much any single training record can influence the final model. Each technique has been studied at length on its own. What is harder to find is a clear, empirical comparison of these techniques used together on the same dataset and the same model.

2.1 Problem Identified and Solution Proposed.

I have identified a specific deficiency in current practice for sensitive-data ML pipelines: when an engineer has to build one, there is no clear empirical guidance on whether to use k-anonymity, differential privacy, both together, or neither. Each choice has a cost in model accuracy and an unclear payoff in privacy, and the existing literature compares the techniques in isolation on different datasets, which makes them hard to weigh against each other in practice. I plan to address this by building one end-to-end secure ML pipeline (encrypted storage, de-identification, training, inference API, audit log) on a single healthcare dataset and a single model architecture, training the model under all four defence configurations (none, k-anonymity only, DP only, both combined), running the same black-box membership inference attack against each, and producing a privacy-utility chart that lets a practitioner read off the trade-off and pick the configuration that fits their constraints.

3. Objectives

The objectives of this dissertation are:

- To design an end-to-end pipeline architecture that handles ingestion, de-identification, privacy-preserving training, controlled inference, and audit logging as separate, testable stages.
- To build a de-identification module that performs attribute classification, masking of direct identifiers, and k-anonymity at $k = 5$ on the chosen dataset, and that stores the raw source records under authenticated encryption.
- To train and evaluate four pipeline configurations on the same data and the same model architecture: no defence (baseline), k-anonymity only, differential privacy only, and both combined. The DP variants will be trained at three privacy budgets ($\epsilon = 1, 3, 8$) using Opacus DP-SGD.
- To deploy the trained models behind a REST API protected by JWT authentication and role-based access control, with every inference request written to a chained-hash audit log in SQLite.

- To run a black-box membership inference attack against every configuration and plot the resulting privacy-utility trade-off, so the value of each defence and the combination can be read off the chart rather than argued in the abstract.
- To produce a STRIDE threat model that links each control in the pipeline to the threat it is meant to address, and to discuss in writing the attacks not run experimentally (model inversion and attribute inference).

4. Scope of Work

The scope is restricted to tabular sensitive data. Specifically, the work uses the UCI Diabetes 130-US Hospitals readmission dataset, which has roughly 100,000 patient encounter records and a binary readmission label. Image, text, and time-series settings are out of scope, as is federated learning. Everything runs on a single host on commodity hardware. Multi-party computation, on-device training, and homomorphic-encrypted inference are mentioned as future work but are not built here.

The pipeline is organised in five stages. Raw records are encrypted at rest using Fernet, which combines AES-128-CBC with HMAC-SHA256 for authenticated encryption. The de-identification stage classifies columns into direct identifiers, quasi-identifiers, sensitive attributes, and non-sensitive attributes, then applies suppression and generalisation to satisfy k -anonymity at $k = 5$. The training stage produces a multilayer perceptron in PyTorch under four configurations: no defence, k -anonymity only, DP only, and both combined. The DP variants are trained with Opacus DP-SGD at epsilon in $\{1, 3, 8\}$ and $\delta = 1e-5$. The trained models are served through a FastAPI service that requires JWT authentication and enforces two roles, analyst and administrator. Every inference request is recorded in a SQLite audit log where each row is chained to the previous one by hash, so any tampering with the log can be detected after the fact.

The piece of the work that gives it research weight, rather than just engineering, is the four-way comparison. All four configurations are evaluated on two axes: predictive utility, measured by accuracy, F1 and ROC-AUC, and resistance to a black-box membership inference attack adapted from Shokri et al. The output is a privacy-utility Pareto frontier and a clear answer about whether layering the two defences buys real protection or just costs accuracy. A STRIDE threat model accompanies the experiments, mapping each pipeline control to the threat it is designed to address, and a separate qualitative section discusses attacks that are not run experimentally.

4.1 Form of the deliverable.

The project is delivered as an integrated Python application, not as a single notebook. Each pipeline stage (encryption, de-identification, training, inference API, audit log) is implemented as its own Python module with a documented interface, and the modules are wired together through a configuration-driven orchestrator. The whole system is version-controlled in a Git repository, has unit tests for each module and an end-to-end integration test that exercises the full flow from encrypted raw input through to an authenticated prediction with a corresponding audit log entry. A working demo is producible at any point during the project for inspection by the supervisor or examiner.

5. Tools, Technologies and System Architecture

This section lists the tools used in the project, what each one does, and shows how they fit together.

5.1 Tools and their role.

Tool / Library	Role in the pipeline
Python 3.10	Implementation language for every module in the pipeline.
PyTorch (CPU build)	Defines and trains the multilayer perceptron classifier. CPU build is sufficient because the model and dataset are small.
Opacus	Wraps the PyTorch optimiser to apply DP-SGD: per-sample gradient clipping plus calibrated Gaussian noise, with a privacy budget tracker.
pandas, NumPy, scikit-learn	Data loading, exploratory analysis, train/test splits, baseline logistic regression, and computation of accuracy, F1, and ROC-AUC.
cryptography (Fernet)	At-rest encryption of the raw dataset using AES-128-CBC with HMAC-SHA256 for authenticated encryption.
Custom de-identification module	Classifies attributes, suppresses direct identifiers, and applies generalisation rules to satisfy k-anonymity at $k = 5$. Includes a verifier that confirms the output meets the k-anonymity property.
FastAPI + Uvicorn	Serves the trained model through an HTTP REST endpoint, with input validation through Pydantic.
PyJWT	Issues and verifies JSON Web Tokens carrying user identity and role (analyst or administrator) for the inference API.

Tool / Library	Role in the pipeline
SQLite + chained-hash log	Records every inference request with subject, timestamp, hashed input, and a hash linking back to the previous row, so any tampering breaks the chain.
Membership inference attack module	Custom implementation following Shokri et al. (2017): trains shadow models, builds an attack classifier, and reports attack AUC against each of the four pipeline configurations.
Matplotlib	Generates the privacy-utility Pareto plot, the per-configuration metric tables, and other figures for the dissertation report.
Pytest	Unit tests for each module and an end-to-end integration test that exercises the full pipeline.
Git + GitHub	Version control. The repository will be made available to the supervisor and examiner for review.

5.2 System architecture.

The diagram below shows how the modules fit together. The top row is the production data flow: raw records are stored encrypted, de-identified, used to train the model, and the trained model is served behind an authenticated API with every request written to a tamper-evident audit log. The lower branch is the evaluation flow: the same trained models are also fed into a membership inference attack, and the result is a privacy-utility comparison across the four configurations.

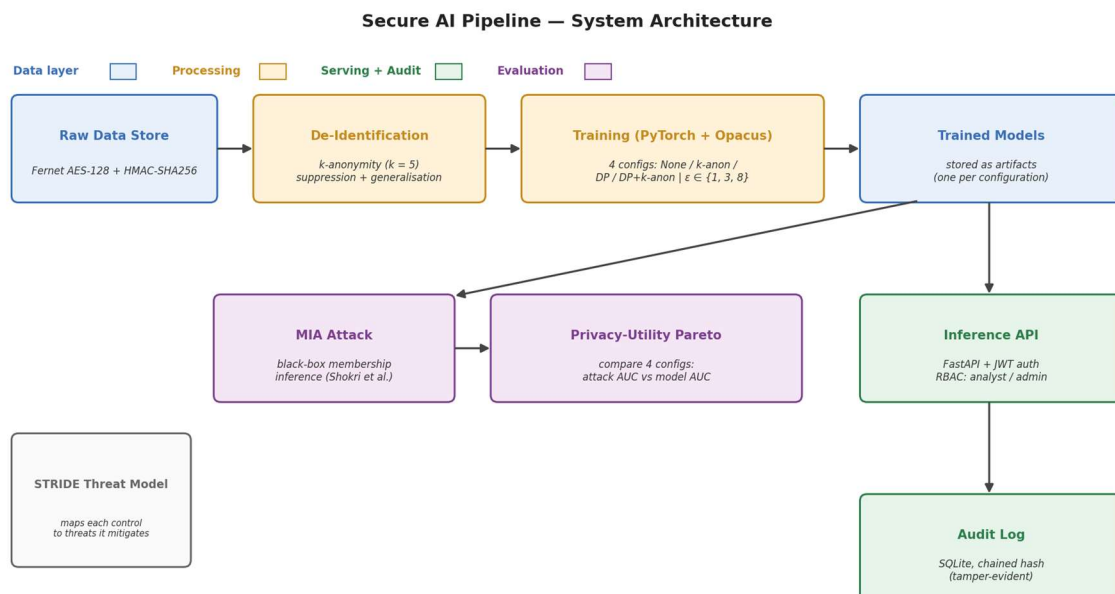


Figure 1: Secure AI Pipeline architecture. Modules are colour-coded by layer (data, processing, serving, evaluation).

6. Plan of Work and Measurable Milestones

The work runs across the NSP-4 Second Semester 2025-2026 calendar (25 April 2026 to 28 August 2026). Effort is roughly fourteen hours per week. Rather than describe progress in generic terms ("design complete", "development complete"), this plan defines specific demoable artifacts that must exist by each evaluation point. Each artifact below can be exercised live during the corresponding VIVA.

6.1 Phase plan.

Phase	Start Date – End Date	Work to be done
Outline Submission	05 May 2026 – 10 May 2026	Get supervisor sign-off. Submit outline on the portal.
Lit Review & Abstract VIVA	11 May 2026 – 29 May 2026	Read 12–15 papers. Set up Python environment. Set up Git repo. Slides + Abstract VIVA.
Dataset & EDA	30 May 2026 – 05 Jun 2026	Download UCI Diabetes dataset. Exploratory analysis. Tag columns as identifier, quasi-identifier, sensitive, or non-sensitive.
Encryption + De-id + Configs 1–2	06 Jun 2026 – 15 Jun 2026	Build encryption module (Fernet). Build de-id module ($k = 5$) with verifier. Train baseline (Config 1) and k-anonymity-only (Config 2) models.
Mid-Sem Report + Demo	16 Jun 2026 – 21 Jun 2026	Write mid-semester report. Plagiarism check. Package the four mid-sem demo pieces (see below) into a runnable form.
Mid-Sem VIVA + DP Configs 3–4	22 Jun 2026 – 10 Jul 2026	Attend Mid-Sem VIVA. Train Config 3 (DP only) and Config 4 (DP + k-anonymity) at $\epsilon = 1, 3, 8$ with Opacus DP-SGD.
Inference API + Audit Log	11 Jul 2026 – 20 Jul 2026	Build FastAPI service. JWT auth. RBAC (analyst, administrator). Chained-hash audit log in SQLite. Integration tests.
MIA Attack + Pareto Plot + STRIDE	21 Jul 2026 – 27 Jul 2026	Run membership inference attack on all four configurations. Plot privacy-utility Pareto frontier. Write STRIDE threat model.
Final Report + Plagiarism	28 Jul 2026 – 02 Aug 2026	Finish dissertation. Plagiarism check. Submit final report.
Final VIVA Defence	03 Aug 2026 – 28 Aug 2026	Record demo video. Make slides. Run integrated end-to-end demo. Defend at Final VIVA.

6.2 Measurable milestones — Mid-Semester VIVA (target by 21 June 2026).

By the time the mid-semester report is submitted, the following four artifacts exist and can be demonstrated live:

- Demo Piece 1 — Encryption module. A command-line tool that takes a raw CSV, encrypts it with Fernet, and stores it in the project's encrypted store. A second invocation decrypts it on demand using the key. Demoable as: run the encrypt command, show the on-disk file is unreadable, run decrypt, show the original CSV reappears.
- Demo Piece 2 — De-identification module with k-anonymity verifier. Takes a CSV in, applies suppression and generalisation, writes a de-identified CSV out, then runs the verifier to confirm every quasi-identifier group has at least $k = 5$ records. Demoable as: run the script on the dataset, show the verifier passes, open the output CSV to show generalised columns.
- Demo Piece 3 — Baseline classifier (Config 1). PyTorch MLP trained on the original dataset. Reports accuracy, F1, ROC-AUC on the held-out test set. Demoable as: run the training script, show the live progress, show final metrics on screen.
- Demo Piece 4 — k-anonymity-only classifier (Config 2). Same model, trained on the de-identified output of Demo Piece 2. Demoable as: run training, show metrics, place side-by-side with Demo Piece 3 to show the utility cost of k-anonymity alone.

6.3 Measurable milestones — Final VIVA (target by 02 August 2026).

By the final report submission, the system is end-to-end and the following additional artifacts exist:

- Demo Piece 5 — DP-only classifier (Config 3) and combined DP + k-anonymity classifier (Config 4), trained at $\epsilon = 1, 3, 8$. Output: a results table with accuracy, F1, ROC-AUC for all four configurations across all privacy budgets where applicable (seven trained models in total).
- Demo Piece 6 — Inference API live demo. Start the FastAPI server, request a JWT for the analyst role, send a prediction request with a curl call, get a response. Show that a request without a JWT or with the wrong role is rejected. Show the corresponding audit log entry being written.
- Demo Piece 7 — Tamper-evident audit log demo. Modify one row of the SQLite audit log directly. Run the chain verifier. Show that the verifier detects the tampering and reports the row where the chain breaks.
- Demo Piece 8 — Membership inference attack and Pareto plot. Run the attack against all four configurations, print the attack AUC for each, generate the privacy-utility Pareto chart as a PNG, and discuss what it shows.

- Demo Piece 9 — Integrated end-to-end run. A single command that takes the encrypted raw dataset, decrypts it, de-identifies it, trains the configured model, starts the API, and makes one authenticated prediction with an audit log entry. This is the proof that the pipeline is one integrated system, not nine separate scripts.
- Demo Piece 10 — Final dissertation report submitted, with the STRIDE threat model chapter mapping every pipeline control to the threats it mitigates, the qualitative discussion of attacks not run experimentally (model inversion, attribute inference), and full reproducibility instructions for the supplied Git repository.

7. Literature References

The following references constitute the preliminary literature reviewed for the current project, drawn from privacy-preserving machine learning, differential privacy theory, and applied healthcare data security.

- [1] Abadi, M., Chu, A., Goodfellow, I., McMahan, H. B., Mironov, I., Talwar, K., and Zhang, L. (2016). *Deep Learning with Differential Privacy*. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 308–318.
- [2] Sweeney, L. (2002). *k-Anonymity: A Model for Protecting Privacy*. *International Journal on Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5), 557–570.
- [3] Dwork, C., and Roth, A. (2014). *The Algorithmic Foundations of Differential Privacy*. *Foundations and Trends in Theoretical Computer Science*, 9(3-4), 211–407.
- [4] Yousefpour, A., Shilov, I., Sablayrolles, A., Testuggine, D., Prasad, K., Malek, M., Nguyen, J., Ghosh, S., Bharadwaj, A., Zhao, J., Cormode, G., and Mironov, I. (2021). *Opacus: User-Friendly Differential Privacy Library in PyTorch*. *arXiv preprint arXiv:2109.12298*.
- [5] Shokri, R., Stronati, M., Song, C., and Shmatikov, V. (2017). *Membership Inference Attacks Against Machine Learning Models*. *Proceedings of the 2017 IEEE Symposium on Security and Privacy*, 3–18.
- [6] Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., and Clore, J. N. (2014). *Impact of HbA1c Measurement on Hospital Readmission Rates: Analysis of 70,000 Clinical Database Patient Records*. *BioMed Research International*, Article 781670.
- [7] McMahan, H. B., Moore, E., Ramage, D., Hampson, S., and Aguera y Arcas, B. (2017). *Communication-Efficient Learning of Deep Networks from Decentralized Data*. *Proceedings of AISTATS*, 1273–1282.
- [8] Government of India (2023). *The Digital Personal Data Protection Act, 2023*. Ministry of Electronics and Information Technology.

- [9] Machanavajjhala, A., Kifer, D., Gehrke, J., and Venkatasubramanian, M. (2007). *l-Diversity: Privacy Beyond k-Anonymity*. *ACM Transactions on Knowledge Discovery from Data*, 1(1).
- [10] Fredrikson, M., Jha, S., and Ristenpart, T. (2015). *Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures*. *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 1322–1333.

8. Particulars of the Supervisor and Examiner

	Supervisor	Additional Examiner
Name	ROSHINI V	NIVEDITHA D S
Qualification	Masters in Computer Applications	Masters in Computer Applications
Designation	Associate - Products Analyst	Associate - Data
Employing Organization and Location	Cognizant Technology Solutions, Chennai	Cognizant Technology Solutions, Bangalore
Phone No. (with STD Code)	+91 9486747515	+91 8310360788
Email Address	roshini.vijay2601@gmail.com	niveditha1998.sp@gmail.com

9. Remarks of the Supervisor

Dhirajsing has chosen a problem that is both timely and well scoped for an M.Tech. dissertation. Privacy-preserving machine learning is an area of growing relevance in industry, and the framing here, which combines DP with k-anonymity rather than treating them as alternatives, gives the work a concrete contribution beyond pure implementation. I am satisfied with the proposed approach and approve this project.

		
Name: RAJPUT DHIRAJ SING RAJESH SING	Name: ROSHINI V	Name: NIVEDITHA D S